

Class 10

Views, Optimization & Real-World Query Practices

◆ 1. Views — Virtual Tables

A **View** is a **saved PostgreSQL query** that behaves like a table.

It doesn't store actual data — only the query definition.

When you query a view, PostgreSQL dynamically fetches the results.

Think of it as:

| “ A “window” that lets you look at complex data easily, without rewriting long queries.

Create a View

```
CREATE VIEW employee_summary AS  
SELECT name, department, salary  
FROM employees  
WHERE salary > 50000;
```

This creates a **virtual table** containing only employees with salary above ₹50,000.

Use the View

```
SELECT * FROM employee_summary;
```

Drop the View

```
DROP VIEW employee_summary;
```

Why use Views?

- Simplify complex queries.
- Enhance security (hide certain columns).
- Provide data abstraction (users don't see table structure).

◆ 2. Updatable Views

Some views allow data modification (if based on a single table, no aggregation, etc.).

Example:

```
UPDATE employee_summary  
SET salary = 60000  
WHERE name = 'Aditi';
```

Note

Views with joins, group functions, or subqueries **cannot be updated** directly.

◆ 3. Optimization Concepts

Optimizing queries improves **speed, scalability, and performance** — essential for real-world databases.

a) Index Usage

Indexes make searches faster (as discussed earlier).

“ Use indexes on columns frequently used in **WHERE, JOIN, or ORDER BY**.

```
CREATE INDEX idx_name ON students(name);
```

b) Limit the Columns

Instead of:

```
SELECT * FROM employees;
```

Use:

```
SELECT name, salary FROM employees;
```

Fetch only the required data to improve performance.

c) Use **WHERE** with Care

Avoid unnecessary full-table scans:

```
SELECT * FROM orders WHERE status = 'Completed';
```

Pro Tip:

Combine multiple conditions logically and use indexed columns.

d) Avoid Subquery Overload

Use **JOINS** or **CTEs** instead of deeply nested subqueries when possible — they perform better and are more readable.

e) Analyze Execution Plan

Use:

```
EXPLAIN SELECT * FROM employees WHERE dept_id = 2;
```

Shows how postgresQL executes the query (for optimization insight).

◆ 4. Real-World Query Practices

Here are some practical postgresSQL scenarios based on everything you've learned

a) Employee with Highest Salary in Each Department

```
SELECT department, name, salary
FROM employees e
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE department = e.department
);
```

b) Department-Wise Average Salary

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

c) Recent 5 Orders

```
SELECT * FROM orders  
ORDER BY order_date DESC  
LIMIT 5;
```

d) Customers with No Orders (LEFT JOIN)

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department  
ORDER BY avg_salary DESC;
```

◆ 5. Database Security & Best Practices

```
GRANT SELECT, INSERT ON employees TO 'user1'@'localhost';
```

1. **Grant specific privileges only**
2. **Use Views** to restrict sensitive data access.
3. **Backup regularly** using `mysqldump` or similar tools.
4. **Validate inputs** in your backend code to prevent PostgreSQL Injection.

◆ 6. SQL Project Integration Ideas (for Developers)

Here are some real-world ideas you can use your SQL knowledge for:

- **Banking System:** Account balance + transactions with rollback logic
- **E-commerce Database:** Products, users, orders, reviews (joins + foreign keys)
- **Chat Application Backend:** Messages, users, timestamps (views + indexing)
- **Analytics Dashboard:** Aggregates, CTEs, and window functions for insights